



Addis Ababa University

College of Natural and Computational Sciences

Department of Computer Science

Programming and Algorithms Module



Part I: Computer Programming

Part II: Object Oriented Programming

Part III: Design and Analysis of Algorithms

Part IV: Data Structure and Algorithms

May 2024
Addis Ababa,
Ethiopia

1. Introduction to programming

Computer programming is the process of writing, testing, debugging/troubleshooting, and maintaining the source code of computer programs. Programs are the instructions that tell the computer what to do. So, writing computer programs means writing instructions, that will make the computer follow and run a program based on those instructions. We are going to write computer programs to solve problems.

Problem Solving

Problem: gap between the existing and the desired or proposed situation where problem solving will try to fill this gap.

Problem solving: The process of transforming the description of a problem into the solution by using our knowledge of the problem-solving strategies, techniques and tools.

Problem solving techniques

- Analyze the problem: Carefully understand the problem
- Outline the problem requirements
 - Does program require user interaction?
 - Does program manipulate data?
 - What is the output?
- Design steps (algorithm) to solve the problem

Algorithm is an ordered set of unambiguous, executable steps, defining a terminating process.

Important Properties of Algorithms

- **Correct:** always returns the desired output for all legal instances of the problem.
- **Unambiguous:** Each step must be clearly defined, having only one interpretation.
- **Precise:** simple but efficient
- **Efficient:** can be measured in terms of Time and Space
- **Finiteness:** the execution of the algorithm must terminate in a finite amount of time for each possible set of input data/must have an end.

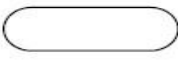
The three commonly used tools to help to document program logic (the algorithm) are flowcharts, structured chart, and Pseudocode.

Pseudocode: is like a programming language but its rules are less inflexible and Written as a combination of English and these programming constructs. It is used for larger problems

Structured chart: depicts the logical functions to the solution of the problem using a chart.

Flowcharts: uses different symbols to represent different processes. It works well for small problems.

Symbols used in flow chart

Symbol	Name	Function
	Process	Indicates any type of internal operation inside the Processor or Memory
	input/output	Used for any Input / Output (I/O) operation. Indicates that the computer is to obtain data or output results
	Decision	Used to ask a question that can be answered in a binary format (Yes/No, True/False)
	Connector	Allows the flowchart to be drawn without intersecting lines or without a reverse flow.
	Predefined Process	Used to invoke a subroutine or an interrupt program.
	Terminal	Indicates the starting or ending of the program, process, or interrupt program.
	Flow Lines	Shows direction of flow.

Programming Language: set of written symbols that instruct computer hardware to perform specified operations. It defined by:

Syntax: a set of rules that governs use of those symbols in programming language

Semantics: deal with the meanings given to syntactically correct constructs of the language.

Categories of programming language: -

Low-level languages: -

- are machine-dependent
- They are designed to be run on a particular computer (not portable).

Eg. Machine language and Assembly Language

High-level languages

- Are machine-independent
- Can be run on a variety of computers.

Programming language require translation source code to machine language before execution. This translation is accomplished by either a compiler or an interpreter. Compilers translate the entire source

code program before execution. Interpreters translate source code programs one line at a time. Interpreters are more interactive than compilers.

programming paradigm: provides the programmer's view of code execution. The most influential paradigms are:

Procedural Programming Languages

Procedural programming specifies a list of operations that the program must complete to reach the desired state. Each program has a starting state, a list of operations to complete, and an ending point. This approach is also known as imperative programming.

Structured Programming Languages

Structured programming is a special type of procedural programming. It provides additional tools to manage the problems that larger programs were creating. Structured programming requires that programmers break program structure into small pieces of code that are easily understood. One of the well-known features of structural programming is that it does not allow the use of the GOTO statement. It is often associated with a "top-down" approach to design.

Object-Oriented Programming Languages

Object-oriented programming is one the newest and most powerful paradigms. In object- oriented programs, the designer specifies both the data structures and the types of operations that can be applied to those data structures.

Basics of programming

The compilation process of C++ program consists of the following steps.

- edit source code → compile → link → execute
(Editor) (Compiler) (Linker) (Loader)
 - I. Program source code is entered into a file using a text editor.
 - II. After the code has been entered, a *compiler* program is started that translates the source into an ***object code file***.
 - III. The object code file is ***linked*** with other **object code files** that come with the compiler and an ***executable file (or program)*** is created.
 - IV. In order to execute the program, a program called the ***loader*** copies the executable file into the memory of the computer and sends an execute command to the CPU.

C++ Basics Elements

1. **Keywords (reserved words)**

- Unique meaning within a C++ program and must not be used for any other purposes. E.g., if, namespace for, switch, continue, float, new, default, break Etc.

2. **Identifiers**

- Name associated with a function or data object and used to refer to that function or data object. An identifier must:
 - o Start with a letter or underscore
 - o Consist only of letters, the digits 0-9, or the underscore symbol _
 - o Not be a reserved word

Example of valid identifiers

_age, age, work10,

Example of non-valid identifiers

E.g.int, main,10age

3. **Literals**

Literals are constant values which can be a number, a character or a string.

E.g. For example, the number 10.5, the character 'A' and the string "Good Afternoon" are all literals

4. **Comments**

A comment is a piece of descriptive text which explains some aspect of a program. Program comments are totally ignored by the compiler and are only intended for human readers.

Two types of comment

- Single line comment (//):
- Multiple line comment (/* */):

5. **Data Types, Variables, and Constants**

Variable:

- Variable is a symbolic name for a memory location in which data can be stored.
- Attribute of Variables:
 - o A type, which is, established when the variable is defined (e.g., integer, float, character)
 - o A value, which can be changed by assigning a new value to the variable

Variable Declaration

- Declaring a variable means defining (creating) a variable.
- Syntax: data type variable name; e.g. int num;
- We can create More Than One Variable at a Time e.g., int num1, num2, num3;

- Assigning Values to Variables e.g., `int num; num=5;`

Scope of Variables

```
#include<iostream>

Using namespace std;

int num;
char ch;
double x;

int main()
{
    float y;
    int age;

    cout<<"Enter your age";
    cin>>age;
    ...
}
```

Global variable

Local variable

Instructions

Global variables can be referred anywhere in the code, within any function, whenever it is after its declaration

The scope of the local variables is limited to the code level in which they are declared.

6. Communication through console cout (the standard output stream)

- used to display information for the user

Syntax: `cout<<object;`

- Object can be variable or literals

Example: `int x=5; cout<<x`
`cout<< "Hello there";`

cin(the standard input stream):

- used to accept information from the user.

Syntax:

`cin>>variable name;`

Eg. `cin>>x;`

Example:

```
// A program that accepts two numbers from the user and swap its
//contents without using a third variable
#include<iostream>
using namespace std;
int main()
```

Sample output

```

{
    int num1, num2;
    cout<<"Enter two number:\n";
    cin>>num1>>num2;
    cout<<"Numbers before Swapping:"<<endl;
    cout<<"num1="<<num1<<"\t"<<"num2="<<num2<<endl;
    num1=num1+num2;
    num2=num1-num2;
    num1=num1-num2;
    cout<<"Numbers After Swapped:"<<endl;
    cout<<"num1="<<num1<<"\t"<<"num2="<<num2<<endl;
    return 0;
}

```

```

Enter two number:
10
20
Numbers before Swapping:
num1=10 num2=20
Numbers After Swapped:
num1=20 num2=10

```

7. Operators

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations.

Assignment Operators (=)

Example: `int x=5;` or used with a combination of other operators' example, `x+=2;` `x=x+2`, `x=y=10;`

Arithmetic Operators (+, -, *, /, %)

E.g. Assume `int X=9, Y=2` `X+Y, Y-X, X%Y`

Relational operators (==, !=, <, <=, >, >=)

Relational operators evaluate to 1 (representing the true outcome) or 0 (representing the false outcome).

Assume `int X=9, Y=2` `X==Y, X!=Y, X>Y`

`'A' < 'F'` // gives 1 (is like `65 < 70`)

Logical operators (! Negation, &&, ||)

Like the relational operators, logical operators evaluate to 1 or 0.

Example: `5 < 6 || 6 < 5` // gives 1

In general, any nonzero value can be used to represent the logical true, whereas only zero represents the logical false

`!12` // gives 0
`10 && 5` // gives 1

Increment/decrement Operators

The auto increment (`++`) and auto decrement (`--`) operators provide a convenient way of, respectively, adding and subtracting 1 from a numeric variable.

Both operators can be used in prefix and postfix form.

Ternary /Conditional operator (?:)

A ternary operator evaluates the test condition and executes a block of code based on the result of the condition.

Syntax: Condition? Expression 1: Expression2

If the condition is true expression 1 will be evaluated otherwise Expression 2 will be evaluated.

Example on operators

<pre>//What will be the output of the following program? #include <iostream> using namespace std; int main() {int a=10,b=3,c,d,e,f; c=a%b++; cout<<"c="<<c<<endl; b+=2; cout<<"b="<<b<<endl; d=++a+b; cout<<"d="<<d<<endl; e=a>b?a:b; e++; cout<<"e="<<e++<<endl; f=a+b+c+d+e; cout<<"f="<<f; return 0; }</pre>	Output: c=1 b=6 d=17 e=12 f=48
--	--

Constant: A constant is any expression that has a fixed value like integer, float, character and string

Eg. 120, 15.2, 'A', "Hello There"

Generally, we have two ways to define constant:

1. Using Const Key word
Syntax: const type constant name=value; e.g. const float PI=3.14;
2. Using #define Keyword
Syntax: #define constant name value e.g. #define PI 3.14

<pre>//a program that accepts a radius of a circle //from the keyboard //and display its area. #include<iostream> #include<math.h> using namespace std; #define PI 3.14 int main() { double radius,area;</pre>	Sample output: Enter the radius of a circle: 10 Area of the circle is:314
--	--

```

    cout<<"Enter the radius of a circle:\n";
    cin>>radius;
    area=PI*pow(radius,2);
    cout<<"Area of the circle is:"<<area;
    return 0;
}

```

Control Statements

Control statements or flow control determines what is executed during a run and what is not, therefore affecting the overall outcome of the program. C++ provides different forms of statements for different purposes. Declaration statements are used for defining variables. Assignment-like statements are used for simple, algebraic computations. Branching statements are used for specifying alternate paths of execution, depending on the outcome of a logical condition. Loop statements are used for specifying computations, which need to be repeated until a certain logical condition is satisfied

Conditional Statements

A. The if Statement

Syntax: *if (expression)*
statement;

First *expression* is evaluated. If the outcome is nonzero (true) then *statement* is executed. Otherwise, nothing happens.

if-else statement

Syntax: *if (expression)*
statement₁;
 else
statement₂;

First expression is evaluated. If the outcome is nonzero (true) then *statement₁* is executed. Otherwise, *statement₂* is executed.

Example

```

//a program that accepts a number from keyboard and display whether
//the numbers is even or odd
#include <iostream>
using namespace std;

```

Sample Output

```

Enter a positive Number:23
The Number is Odd

```

<pre> int main() { int n; cout<<"Enter a positive Number:"; cin>>n; if(n%2==0) cout<<"The Number is Even"; else cout<<"The Number is Odd"; return 0; } </pre>	
---	--

if-else if statement

Syntax: if (*expression1*)

statement1;

else if (*expression2*)

statement2;

else if (*expression3*)

statement3;

else

statement 4

First *expression1* is evaluated. If the outcome is nonzero (true) then *statement1* is executed. Otherwise, *expression2* is evaluated. If the outcome is nonzero (true) then *statement2* is executed. Otherwise, *expression3* is evaluated. If the outcome is nonzero (true) then *statement3* is executed. Otherwise *statement4* is executed.

switch Statement

The switch statement provides a way of choosing between a set of alternatives, based on the value of an expression. The general form of the switch statement is:

```

switch (expression) {
    case constant1:
        statements;
    ...
    case constantn:
        statements;
    default:
        statements;
}

```

First *expression* (called the switch **tag**) is evaluated, and the outcome is compared to each of the numeric *constants* (called case **labels**), in the order they appear, until a match is found. The *statements* following the matching case are then executed.

Iteration/Looping Statement:

Iteration is the repetition of a statement or block of statements in a program.

- *Loops* have as objective to repeat a *statement* a certain number of times or while a condition is fulfilled.
- C++ has three iteration statements: the `while` statement, the `do. While` statement, and the `for` statement.

The *while* loop:

Its format is:

while (*expression*) {*statement*;}

First *expression* (called the **loop condition**) is evaluated. If the outcome is nonzero then *statement* (called the **loop body**) is executed and the whole process is repeated. Otherwise, the loop is terminated.

Example

Write a program that will display the sum of the following Series $\text{Sum} = 1 + 2 + 3 + \dots + n$, for an input integer n .

```
#include <iostream>
using namespace std;
int main()
{ int n, i=1;
  cout << "Enter a positive integer: ";
  cin >> n;
  int sum=0;
  while (i <= n)
  {
    sum+= i++;
  }
  cout << "The sum of the first "<< n<<
  "integers is:"<< sum;
      return 0;
}
```

Sample output

```
Enter a positive integer: 5
The sum of the first 5integers is:15
```

The *do-while* loop

The condition in the *do-while* is evaluated after the execution of statement instead of before, granting at least one execution of statement even if condition is never fulfilled.

do {statement; }while (condition);

First *statement* is executed and then *condition* is evaluated. If the outcome of the latter is nonzero then the whole process is repeated. Otherwise, the loop is terminated.

Example

```
#include <iostream>
using namespace std;
int main()
{ int num=1;
do{ cout<<num<<" ";
    num++;
}while (num <=10);
return 0;
}
```

Output

1 2 3 4 5 6 7 8 9 10

The *for* loop.

Its format is:

for (initialization; condition; increase) statement;

1, *initialization* is executed. Generally, it is a initial value setting for a counter variable. This is executed only once

2, *condition* is checked, if it is **true** the loop continues, otherwise the loop finishes and *statement* is skipped.

3, *statement* is executed. As usual, it can be either a single instruction or a block of instructions enclosed within curly brackets { }.

4, finally, whatever is specified in the *increase* field is executed and the loop gets back to step 2.

Example:

```
#include<iostream>
using namespace std;
int main()
{char ch,ch2;
for(ch='A',ch2='a';ch<='G' || ch2<='g';ch++,ch2++)
    cout<<ch<<ch2<<" ";
}
```

Output:

Aa Bb Cc Dd Ee Ff Gg

More than One Control Variable in a for Loop is possible

The **for** loop in this program uses two control variables:

```
int main()
{ for (int m=95, n=11; m%n > 0; m -= 3, n++)
    cout << m << "%" << n << " = " << m%n << endl;
}
```

```
95%11 = 7
92%12 = 8
89%13 = 11
86%14 = 2
83%15 = 8
```

Nested Loop

Nested loops used to evaluate another looping statement inside its body.

Example1:

<pre>#include <iostream> using namespace std; int main() { for (int i=1;i<=5;i++) {for(int j=1;j<=i;j++) cout<<j<<" "; cout<<endl; } return 0; }</pre>	Output <pre>1 1 2 1 2 3 1 2 3 4 1 2 3 4 5</pre>
--	--

Example 2:

<pre>#include<iostream> using namespace std; int main() { int t=0,a,b; for (a=1; a<=5; a+=1) for (b=1; b<=a; b+=2) t += a + b; cout<<t; return 0; }</pre>	Output: <pre>51</pre>
---	------------------------------

Using continue and break Statements

- The **break** statement skips the rest of the statements in the loop's block, jumping immediately to the next statement outside of the loop.
- The **continue** statement is similar. It also skips the rest of the statements in the loop's block, but instead of terminating the loop, it transfers execution to the next iteration of the loop. It continues the loop after skipping the remaining statements in its current iteration.

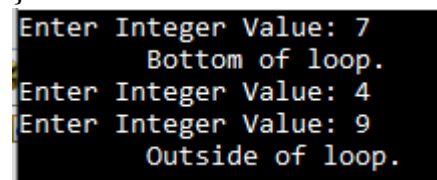
Small program illustrates the **continue** and **break** statements:

```
#include <iostream>
using namespace std;
int main()
```

```

{ int n;
for (;;)
{ cout << "Enter Integer Value: ";
cin >> n;
if (n%2 == 0)
continue;
if (n%3 == 0)
break;
cout << "\tBottom of loop.\n";
}
cout << "\tOutside of loop.\n";
return 0;
}

```



```

Enter Integer Value: 7
    Bottom of loop.
Enter Integer Value: 4
    Bottom of loop.
Enter Integer Value: 9
    Outside of loop.

```

C++ goto Statement

Syntax:

```
goto label1;
```

```
statements
```

```
----
```

```
----
```

```
Label1:
```

```
statements
```

Function

A function is a group of statements that structure our programs in a more modular way and executed when it is called from some point of the program. It is a block of code that performs a specific task. It is important to Dividing a complex problem into smaller chunks makes our program easy to understand and reusable.

When we need function?

- When you need to repeat the same process over and over in a program.
- The function can be called many times but appears in the code once

Function – Categories

C++ functions can be divided into 2 categories:

• pre-defined (standard function) / library function

- Which is the definitions have been written and it is ready to be used.
- User needs to include pre-defined header file (i.e., math.h, time.h,cmath)

To use it: FunctionName (value) e.g., sqrt (4), floor (4.3)

The Standard Math Library

- fabs : absolute value of floating point number, #include<cstdlib> to call abs() for integer number
- floor: largest integral value not greater than x

- ceil : smallest integral value not less than x
- fmod : floating-point remainder function
- log : logarithm
- pow : Raise a number by a power.
- sin : The sine of an integer.
- sqrt : Square root of a number.
- tan : Tangent.
- tanh : Hyperbolic tangent

Example

```
#include<iostream>
#include <cmath>
using namespace std;
int main()
{double x=-25;
float y=36.5;
int z=25;
cout<<"absolute value:"<<fabs(x)<<endl;
cout<<floor(y)<<endl;
cout<<ceil(y)<<endl;
cout<<sqrt(z);
    return 0;
}
```

output

```
Absolute value:25
36
37
5
```

• User-defined

- Function that been created by the user.
- These functions need to be declared and defined by the user.

Declaration of function

Function defines using the following syntax:

Function_type function_name (Parameter List)

```
{ ...function body
}
```

Example: void Addition(int num1,int num2)

```
#include<iostream>
using namespace std;
void Addition(int num1,int num2)
{    int sum=num1+num2;
    cout<<"The sum is:"<<sum;
}
int main()
{    int x=10,y=20;
    Addition(x,y);
    return 0;
}
```

Passing by Value

In call by value, the actual value that is passed as argument is not changed after performing some operation on it. It creates a copy of that variable into the stack section in memory. When the value is changed, it changes the value of that copy, the actual value remains the same.

Example:

```
#include<iostream>
using namespace std;
void my_function(int x) {
    x = 50;
    cout << "Value of x from my_function: " << x << endl;
}
int main() {
    int x = 10;
    my_function(x);
    cout << "Value of x from main function: " << x;
}
```

Passing by Reference: It allows a function to modify a variable without having to create a copy of it. We have to declare reference variables. The memory location of the passed variable and parameter is the same and therefore, any change to the parameter reflects in the variable as well.

Example:

<pre>//C++program to swap two numbers using pass by reference #include <iostream> using namespace std; void swap(int& x, int& y) { int z = x; x = y; y = z; } int main() { int a = 45, b = 35; cout << "Before Swap\n"; cout << "a = " << a << " b = " << b << "\n"; swap(a, b); cout << "After Swap with pass by reference\n"; cout << "a = " << a << " b = " << b << "\n"; }</pre>	Before Swap a = 45 b = 35 After Swap with pass by reference a = 35 b = 45
--	--

Default Parameters

A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the calling function doesn't provide a value for the argument.

However, if arguments are passed while calling the function, the default arguments are ignored.

All default parameters must be the **rightmost** parameters of the function.

Example:

<pre>//a Program to demonstrate Default Arguments #include <iostream> using namespace std;</pre>	Output: 25 50
--	----------------------------

<pre> int sum(int x, int y, int z = 0, int w = 0) { return (x + y + z + w); } int main() { cout << sum(10, 15) << endl; cout << sum(10, 15, 25) << endl; cout << sum(10, 15, 25, 30) << endl; return 0; } </pre>	80
--	----

Scope of variables

Scope of variables defined as the extent of the program code within which the variable can be accessed or declared or worked with

mainly two types of variable scopes:

local variable:

- Variables defined within a function or block are said to be local to those functions
- Declared inside a block{ }

Global Variable

- variable that can be accessed from any part of the program.
- They are available throughout the life time of a program.
- They are declared at the top of the program outside all of the functions or blocks.

NB:

- Usually when two variables with same name are defined then the compiler produces a compile time error. But if the variables are defined in different scopes, then the compiler allows it.
- Whenever there is a local variable defined with same name as that of a global variable then the *compiler will give precedence to the local variable*

Scope resolution Operator (: :)

If we want to access global variable when there is a local variable with same name in local scope, we use scope resolution operator

<pre> #include <iostream> using namespace std; int num=5,x ; int main() { int num = 20; cout<<"Value of local num is:"<<num; cout <<"\nValue of global num is:"<<::num; cout<<"\nValue of global x:"<<x; } </pre>	<u>Output:</u> Value of local num is:20 Value of global num is:5 Value of global x:0
---	--

<pre> return 0; } </pre>	
--------------------------	--

Function Overloading

A function that can have the same name if the number and/or type of arguments passed is different are known as overloaded functions. If function overloading is done containing the default arguments, then we need to make sure it is not ambiguous to the compiler, otherwise it will throw an error.

Example

<pre> #include <iostream> using namespace std; int sum(int x, int y) { return (x + y); } double sum(double x, double y) { return (x + y); } int main() { cout << sum(10.5, 15.3) << endl; cout << sum(10, 15) << endl; return 0; } </pre>	Output: 25.8 25
---	------------------------------

Recursive Functions

- Functions that call themselves
- A function that is defined in terms of itself is called recursive
- There must be a base case e.g. $0! = 1$
- Slowly converges towards base case
- For example, Recursive functions $N! = N * (N-1)!$

This function can be implemented both iteratively or recursively as

//iterative implementation

```

int factorial (int N)
{
    int fact =1;
    for(int i=1;i<=n; i++)
    {
        fact *=i;
    }
    return fact;
}

```

//recursive implementation

```

int factorial (int n)
{
    if (n == 0) return 1;
}

```

```
        return n*factorial(n-1);
    }
}
```

Arrays

- an **array** is a collection of fixed-size number of elements of the same type stored sequentially in memory.
- An individual element of an array is identified by its own unique **index** (or **subscript**).
- The number of elements **must be an integer**.
- The size of the array is referred to as its dimension
- Array in C++ is Zero indexed.

Declaring of one-dimensional Array

Syntax: elementstype arrayname [size];

➔ You can use constants such as macro definitions or const ints as the sizes of arrays.

Example:

```
int num[5];
```

```
string fruit[3];
```

```
#define numstudents 100          or    const int numstudents=100;
```

```
double Score[numstudents] ;      double Score[numstudents] ;
```

Initialization of an Array

- Arrays can be initialized at declaration time in two different ways:
Syntax:
elementstype arrayname[size] = {list each value separated by comma};
E.g int num[5]={1,2,3,4,5};

elementstype arrayname[] = { list with any number of values };
E.g int num[]={1,2,3,4,5};
string fruit[]={“Apple”,“orange”, “Mango”};

Accessing Array Elements

- Each element could be accessed using an index into the array
- When accessing an array, the index given must be a positive integer from 0 to n-1, where n is the dimension of the array.

The **syntax** is:

```
arrayname[integer-value-within-range];
```

Example:

```
const int SIZE = 100;
```

```
int num[SIZE]={0};
```

```
string fruit[]={“Apple”,“orange”, “Mango”};
```

```

cout<<fruit[0]; //print Apple
fruit[1]= "Lemon"; //replaces "orange" by lemon in fruit[1]
cout<<fruit[1]; //print Orange
num[SIZE-10]=num[SIZE-11]+2; // // sets num[90] to num[89] + 2=2
cout<< num[SIZE-10]; //print 2

```

Example:

Write a program that accepts the score of 5 students from the user and display the minimum score.

```

#include <iostream>
using namespace std;
int main()
{const int size=5;
double score[size];
for( int i = 0 ; i <size; i++ )
{   cout<<"Enter the "<<i+1<<" Element:";
    cin>>score[i];
}
int minscore = score[0];
for( int i =1; i <size; i++)
{ if(minscore>score[i] )
  minscore = score[i] ;
}
  cout<< "The minimum score is:"<< minscore << endl ;
  return 0;
}

```

Array as Parameter of Function

- It is possible to use array as parameter of the function.
- When declaring the function, simply specify the array as a parameter, without a dimension. The array can then be used as normal within the function

Example:

```

#include <iostream>
using namespace std;
int Addition(int array[], int size)
{int s=0;
  for(int i=0;i<size;i++ )
    s=s+array[i];
  return s;
}
int main()
{int num[]={2,3,4,10,15};
cout<<"    The Sum is:"<<Addition(num,5);
  return 0;
}

```

It is important to note that arrays are passed by reference and so any changes made to the array within the function will be observed in the calling scope.

<pre> #include <iostream> using namespace std; int Addition(int array[], int size) {int s=0; for(int i=0;i<size;i++) { array[i]=array[i]*2; s=s+array[i]; } return s; } int main() {int num[]={2,3,4,10,15}; cout<<" The Sum is:"<<Addition(num,5)<<endl; for(int i=0;i<5;i++) cout<<num[i]<<endl; return 0; } </pre>	<pre> Output: The sum is :68 4 6 8 20 30 </pre>
--	---

Multidimensional Array

C++ support the creation of multidimensional array, through the addition of more than one set of brackets.

Syntax: elementstype arrayname [size1] [size2] ... [sizen];

Two-Dimensional Array

Declaration: elementstype arrayname [size1] [size2];

Eg. int num[2][3];//declare 2 row, 3 columns to store 6 elements

Initialization of Two-Dimensional Array

E.g. int num[2][3]={1,2,3,4,5,6};//2 row 3 columns

Or int num[2][3]={{1,2},{3,4},{5,6}};

Example

Write a program that accepts five numbers from the keyboard and display only three of the largest number

<pre> #include<iostream> using namespace std; int main() { int nums[5]; cout<<"Enter the Five elements:\n"; for(int i=0;i<=4;i++) cin>>nums[i]; int temp; </pre>	Sample output: <pre> Enter the Five elements: 9 3 7 6 4 Three of the largest numbers are: </pre>
--	--

<pre> for(int i=0;i<=4;i++) { for(int j=i+1;j<=4;j++) { if(nums[j]>nums[i]) { temp=nums[i]; nums[i]=nums[j]; nums[j]=temp; } } } cout<<"Three of the largest numbers are:"<<endl; for(int i=0;i<=2;i++) { cout<<nums[i]<<" "; } return 0; } </pre>	976
--	-----

String and pointer

String:

- is an ordered sequence of characters
- Sequence of character in which the last character is the null character '\0'
- The null character indicates the end of the string
- C++ provides following two types of string representations:
 - o ***The C-style character string.***
 - o ***The string class type introduced with Standard C++.***

NB:

→ If a string has **n** characters, then it requires an **n+1** element array (at least) to store it.

→ In order to use the string data type, the C++ string header <string> must be included at the top of the program

The C-Style Character String

- originated within the C language and continues to be supported within C++
- array of characters which is terminated by a **null** character '\0'

Example: char status[5]={ 'G','O','O','D','\0'}; //char status[]="Good";

cout<<status;

- C++ supports a wide range of functions that manipulate null-terminated strings:

strcpy(s1, s2); Copies string s2 into string s1

strcat(s1, s2); Concatenates string s2 onto the end of string s1

strlen(s1); Returns the length of string s1.

strcmp(s1, s2); Returns 0 if s1 and s2 are the same; less than 0 if s1<s2; greater than 0 if s1>s2.

strchr(s1, ch); Returns a pointer to the first occurrence of character ch in string s1.

strstr(s1, s2); Returns a pointer to the first occurrence of string s2 in string s1.

```
#include <iostream>
#include <cstring>
using namespace std;
int main ()
{
char s1[15] ="Good";
char s2[15] ="Morning";
char s3[15];
cout<<strcmp(s1,s2)<<endl;
strcpy(s3,s1);
cout<<s3<< endl;
strcat(s1,s2);
cout <<s1 << endl;
cout <<strlen(s1) << endl;
cout<<strchr(s1, 'd')<<endl;
cout<<strstr(s1,s2);
return 0;
}
```

Output:

```
-1
Good
GoodMorning
11
dMorning
Morning
```

The String Class in C++:

- A string is a variable that stores a sequence of letters or other characters
- Example: string course_name="programming";
cout<<course_name
- string manipulations operation

str.length():Counting the number of characters in a string

Appending to a string if you want to append to the end of string, you can use the += operator

Accessing individual characters: You can read and write to characters within a string using []

Comparing two strings: you can compare two strings for equality using the == and !=

Searching within a string str.find(key)

```
#include <iostream>
#include <string>
```

Output:

```

using namespace std;
int main ()
{
string s1 ="Practice Makes Perfect! ice
scream";
string s2="Yes";
string s3;
int n1,n2;
s3=s1+s2;
cout<<s3<<endl;
if(s2=="Yes")
    cout<<"the same string content:"<<endl;
cout<<s1.length()<<endl;
cout<<s1[3]<<endl;
n1=s1.find("ice");
cout<<n1<<endl;
n2=s1.find("ice",n1+1);
cout<<n2<<endl;
return 0;
}

```

Practice Makes Perfect! ice screamYes
the same string content:

34
c
5
24

str.find(key, n) starts at position **n** within **str** and will attempt to find **key** from that point on

Extracting substrings: str.substr(start, length)

topUpperCase, toLowerCase: These functions take a string, and return a new string with all letters in lower or upper case respectively **toLowerCase(orangeFruit)**

There are a few other functions within the **strlib** library (**equalsIgnoreCase, startsWith, endsWith, and trim**) as well.

Example 1

```

#include<iostream>
#include<string>
using namespace std;
int main()
{
    char name[] ="Good Morning Students";
    string message="Great Work";
    cout<<name<<endl;
    cout<<message<<endl;
    cout<<sizeof(name)<<endl;
    cout<<sizeof(message)<<endl;
    cout<<message.length()<<endl;//message.size()<<endl;
    return 0;
}

```

```
}
```

```
Good Morning Students  
Great Work  
22  
4  
10
```

Example2

```
#include<iostream>
#include<cstring>
using namespace std;
int main()
{
    char name[20] ="Hello Students";
    string message="Good Morning";
    char name1[20];
    string l;
    cout<<"enter your name:";
    cin.get(name1,20);// cin.get(name, 20, 's');
    cin.ignore();
    getline(cin,l);
    cout<<name<<endl;
    cout<<message<<endl;
    cout<<sizeof(name)<<endl;
    cout<<sizeof(message)<<endl;
    cout<<message.length()<<endl;//message.size()<<endl;
    cout<<l<<endl;
    cout<<name1<<endl;
    cout<<strlen(name);
    return 0;
}
```

```

enter your name:Lemma Kebede
Amar Tesfaye
Hello Students
Good Morning
20
4
12
Amar Tesfaye
Lemma Kebede
14

```

Pointer

- A *pointer* is a variable that holds a memory address of another variable.
- Its content is a memory address
- A pointer is simply the *address* of a memory location and provides an indirect way of accessing data in memory.
- It is defined to 'point to' data of a specific type.
- Because a pointer variable holds the address of another piece of data, it "points to" the data

Reference operator (&) and Dereference operator (*)

&: read as address of

- Use address operator & to get address of a variable

*****: read as **value pointed by**

Declaring variables of pointer types

Syntax:

type *variable_name;

For example:

```

int *ptr1;      //pointer to an int
double *number;
char *ptr2;     //pointer to a char

```

The *value* of a pointer variable is the address to which it points.

For example, given the definitions

```

int num; we can write: ptr1 = &num;

```

Example 1:

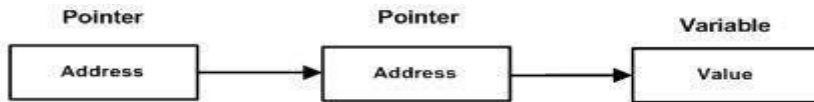
<pre>#include <iostream> using namespace std; int main () { int x =10; int *ptr; ptr = &x; cout << "Address of variable x:"<< &x<< endl; cout << "Address of pointer variable ptr:"<< &ptr<< endl; cout<<"Value stored in x: " << x << endl; cout << "Value stored in ptr variable: " << ptr << endl; cout<<"Value pointed by ptr:"<<*ptr<<endl; return 0; }</pre>	Output <pre>Address of variable x:0x67feec Address of pointer variable ptr:0x67fee8 Value stored in x: 10 Value stored in ptr variable: 0x67feec Value pointed by ptr:10</pre>
---	---

Example 2

<pre>#include <iostream> using namespace std; int main () { int num1 = 5, num2 = 15; int *p1, *p2; p1 = &num1; p2 = &num2; cout<<*p1<<endl; *p1 = 10; cout<<"num1:"<<num1<<endl; *p2 = *p1; cout<<*p2<<endl; p1 = p2; cout<<*p1<<endl; *p1 = 20; cout<<"num1:"<< num1 << endl; cout<<"num2:"<<num2 << endl; cout<<*p2<<" "<<*p1; return 0; }</pre>	Output <pre>5 num1:10 10 10 num1:10 num2:20 20 20</pre>
--	--

A pointer to a pointer is a form of multiple indirection or a chain of pointers

- C++ allows the use of pointers that point to pointers



Syntax: `data_type_of_pointer **name_of_variable = & normal_pointer_variable;`

`int a;`

`int *ptr,**ptr1;`

`ptr=&a;`

`ptr1=&ptr;`

Example:

```

#include <iostream>
using namespace std;
int main () {
    int num,*ptr1,**ptr2;
    num = 15;
    ptr1 = &num;
    ptr2 = &ptr1;
    cout<<"Address of var:"<<&num<<endl;
    cout<<"Address of Ptr1:"<<&ptr1<<endl;
    cout<<"Address of Ptr2:"<<&ptr2<<endl;
    cout << "Value of var :" <<num << endl;
    cout << "Value available at ptr1 :" <<*ptr1 << endl;
    cout<<"ptr2 content:"<<ptr2<<endl;
    cout << "Value available at ptr2 :"<<**ptr2 << endl;
    return 0;
}
  
```

Output:

```

Address of var:0x67feec
Address of Ptr1:0x67fee8
Address of Ptr2:0x67fee4
Value of var :15
Value available at ptr1 :15
ptr2 content:0x67fee8
Value available at ptr2 :15
  
```

void pointers

- Void pointer point to any data type

Example:

`void *ptr;`

`int x=5;`

`Ptr=&x;`

```

#include <iostream>
using namespace std;
int main()
{int a=10;
float b=20;
    void*ptr;
    ptr=&a;
    cout<<*(static_cast<int*>(ptr))<<endl;
    //cout<<*((int*)ptr)
    ptr=&b;
    cout<<*((float*)ptr)<<endl;
}
  
```

Output

```

10
20
0x67fef4
  
```

<pre> cout<<ptr; return 0; } </pre>	
---	--

Pointer arithmetic

- The C++ language allows you to perform integer addition or subtraction operations on pointers.
- If ptr points to an integer, ptr + 1 is the address of the next integer in memory after ptr.
- ptr - 1 is the address of the previous integer before ptr.

Note that ptr + 1 does not return the *memory address* after ptr, but the memory address of the *next object of the type* that ptr points to. If ptr points to an integer (assuming 4 bytes), ptr + 3 means 3 integers (12 bytes) after ptr. If ptr points to a char, which is always 1 byte, ptr + 3 means 3 chars (3 bytes) after ptr.

When calculating the result of a pointer arithmetic expression, the compiler always multiplies the integer operand by the size of the object being pointed to. This is called scaling.

Example:

```

#include <iostream>
using namespace std;
int main()
{
    int x=7;
    int* ptr=&x;
    cout<<"size of int:"<<sizeof(x)<<endl;
    cout<<"Address of ptr:"<<&ptr<<endl;
    cout << ptr <<endl;
    cout << ptr+1<<endl;
    cout << ptr+2<<endl;
    cout << ptr+3<<endl;
    return 0;
}

```

Output:

```

size of int:4
Address of ptr:0x67fef8
0x67fefc
0x67ff00
0x67ff04
0x67ff08

```

Dynamic Memory Allocation

Memory in your C++ program is divided into two parts:

- **The stack** – All variables declared inside the function will take up memory from the stack.
- **The heap** – This is unused memory of the program and can be used to allocate the memory dynamically when program runs.

- We can allocate and then deallocate memory dynamically using the **new** and **delete** operators respectively.
- C++ allows us to allocate the memory of a variable or an array in run time

❖ **Allocating memory in runtime**

Syntax:

pointer = new type

e.g., int*ptr;

ptr=new int;

*ptr=5;//assign value to allocated memory

pointer = new type [number_of_elements];

e.g., int *ptr1;

ptr1=new int[5]

- the first element can be accessed as ptr1[0] or *(ptr1)
- the second element can be accessed as ptr1[1] or *(ptr1+1)

```
double* pvalue = NULL; // Pointer initialized with null
pvalue = new double; // Request memory for the variable
```

❖ **Deallocate the memory occupied by the variable**

Syntax:

delete pointerVariable;

e.g., delete ptr;

delete[]pointerVariable;

e.g. delete []ptr1;

Example:

```
#include <iostream>
using namespace std;
int main()
{   int *ptr,*ptr1;
    ptr=new int;
    ptr1=new int[5];
    *ptr=5;
    *ptr1=2;
    *(ptr1+1)=10;
    cout<<*ptr<<endl;
    cout<<*ptr1<<endl;
    cout<<*(ptr1+1);
    delete ptr;
    delete []ptr1;
    return 0;
}
```

Output:

```
5
2
10
```

Pointer and Array

- Array can bound into a pointer that points to the first element (element 0) of the array.
- An array access can always be implicitly converted to a pointer access of the appropriate type.
For example, consider these two declarations:
`int myarray[20];`
`int *mypointer;`
`Mypointer=myarray;// mypointer=&myarray[0];`
→ Stores the address of the first element of the array in variable mypointer
→ After that, mypointer and myarray would be equivalent and would have very similar properties.
→ The main difference being that mypointer can be assigned a different address, whereas myarray can never be assigned anything, and will always represent the same block of 20 elements of type int.

Therefore, the following assignment would not be valid:

`myarray = mypointer;`

Example 1:

<pre>#include <iostream> using namespace std; int main () { int numbers[5]; int * p; p = numbers; *p = 10; p++; *p = 20; p = &numbers[2]; *p = 30; p = numbers + 3; *p = 40; p = numbers; *(p+4) = 50; for (int n=0; n<5; n++) cout << numbers[n] << ", "; return 0; }</pre>	Output: <pre>10, 20, 30, 40, 50,</pre>
---	---

NB:

→ Generally, `array[n]` is the same as `*(array + n)`, where `n` is an integer.

→ Considering that `int *ptr; int arr[5];`

`ptr = arr;`

`ptr + 1` is equivalent to `&arr[1];`

`ptr + 2` is equivalent to `&arr[2];`

Example 2:

Array name used as pointer

```
// write a C++ Program that accept 5 numbers from
the keyboard and display the numbers using
pointer notation.
#include <iostream>
using namespace std;
int main() {
    float num[5];
    // Insert data using pointer notation
    cout << "Enter 5 numbers:\n ";
    for (int i = 0; i < 5; ++i)
    {
        // store input number in arr[i]
        cin >> *(num + i) ;
    }
    // Display data using pointer notation
    cout << "Displaying data: " << endl;
    for (int i = 0; i < 5; ++i) {
        // display value of arr[i]
        cout << *(num + i) << endl ;
    }
    return 0;
}
```

Output:

```
Enter 5 numbers:
15
3
20
30
45
Displaying data:
15
3
20
30
45
```

Example 3:

```
#include <iostream>
using namespace std;
int main() {
    int num[5]={1,2,3};
    int *p=num;
    // same as *(p++): increment pointer, and
dereference unincremented address
    cout<<*p++ <<endl;
    // same as *(++p): increment pointer, and
dereference incremented address
    cout<<*++p<<endl;
    // same as ++(*p): dereference pointer, and
increment the value it points to
    cout<<++*p<<endl ;
    // dereference pointer, and post-increment the value
it
    cout<<(*p)++<<endl;
    cout<<*p;
    return 0;
}
```

Output:

```
1
3
4
4
5
```

Pointer and function

// the following program demonstrate how to use pointer in functions

```
include<iostream>
using namespace std;
void pointerfun(int *x, int *y);
int main()
{ int a= 5, b=10;
  cout<<"\nOriginal Values\n";
  cout<<"a = "<<a<<" and b = "<<b<<"\n";
  pointerfun(&a, &b);
  cout<<"\nChanged Values\n";
  cout<<"a = "<<a<<" and b = "<<b<<"\n";
  return 0;
}
void pointerfun(int *x, int *y)
{   int a=*y;
    *x = *y*a;
    *y = 20;
}
```

Output:

Original Values
a = 5 and b = 10

Changed Values
a = 100 and b = 20

Pointer and String

Example 1:

```
#include<iostream>
using namespace std;
int main()
{   char name[6]="Hello";
    char *p;
    p=&name[0];
    cout<<p<<endl;
    cout<<*p<<endl;
    cout<<p[1]<<endl;
    cout<<"Display using pointer:\n";
    while(*p!='\0')
    {
        cout << *p;
        p++;
    }
    return 0;
}
```

Output

Hello
H
e
Display using pointer:
Hello

Example 2:

```
#include<iostream>
```

Output:

<pre>using namespace std; int main() { string name="Student Name"; string *p=&name; cout<<name<<endl; cout<<*p; return 0; }</pre>	<pre>Student Name Student Name</pre>
---	--------------------------------------

Example 3:

<pre>#include<iostream> using namespace std; int main() { string name[2]={"Student Name", "Bye"}; string *p; p=name;//p=&name[0] cout<<name[1]<<endl; cout<<*p; return 0; }</pre>	<pre>Output: Bye Student Name</pre>
---	-------------------------------------

Structure in C++

- A structure definition is a user-defined variable type which is a grouping of one or more variables.
- We declare a structure using the keyword struct
- Once the type has been defined through the C++ 'struct' keyword, you can create variables from it just like you would any other type.
- When you create a structure variable based on a structure definition, all of the member variables names are retained.

Syntax for Structure definition:

struct structure name or tag

{ structure members;

};

→ There is no allocated memory at the time of declaration of structure.

→ you can write the structure definition inside the main function

Example:

```
struct student {
```

```
char name[20];
int age;
double score;
};
```

Defining a structure variable

The memory is allocated when the variable of a structure is created.

Syntax:

Structure tage structure variable;

Eg. student st;

Multiple structure variables separated with commas

student st,st1;

Accessing members of a structure variable

A structure member can be accessed using dot operator'.' notation

syntax:

structvariable.member;

eg. st.id=4;

Example1:

<pre>#include<iostream> using namespace std; struct student{ string name; int age; double score; }; int main() {student st; st.name="Abebe"; st.age=19; st.score=24; cout<<"Name:"<<st.name<<endl ; cout<<"Age:"<<st.age<<endl; cout<<"Score:"<<st.score; return 0; }</pre>	Initialize value <pre>#include<iostream> using namespace std; struct student{ string name; int age; double score; }st={"Abebe",19,24}; int main() { cout<<"Name:"<<st.name<<endl ; cout<<"Age:"<<st.age<<endl; cout<<"Score:"<<st.score; return 0; }</pre>	Output <pre>Name:Abebe Age:19 Score:24</pre>
---	---	---

Example 2:

Define structure name called **Rectangle** with structure members **length** and **width**. In your program define function **takeInput** to take inputs from the user and **display** to display the area and perimeter of the rectangle.

```
#include<iostream>
using namespace std;
struct rectangle{
int length;
int width;
}rt;
void takeInput(){
    cout<<"Enter the length and width respectively:\n";
    cin>>rt.length>>rt.width;
}
void display()
{
    cout<<"Area:"<<rt.length*rt.width<<endl;
    cout<<"perimeter:"<<2*(rt.length+rt.width);
}
int main()
{ takeInput();
display();
    return 0;
}
```

Array of structure

Example:

```
#include<iostream>
using namespace std;
struct student{
string name;
int age;
double score;
};
int main()
{student st[5];
cout<<"Enter student information:\n";
for(int i=0;i<5;i++)
{cout<<"enter the"<<i+1<<"stud name:";
cin>>st[i].name;
cout<<"enter the"<<i+1<<"stud age:";
cin>>st[i].age;
cout<<"enter the"<<i+1<<"stud score:";
cin>>st[i].score;
}
```

```

cout<<"Dispaly Student Information:\n";
cout<<"Name   age   score:\n";
for(int i=0;i<5;i++)
{
cout<<st[i].name<<"\t"<<st[i].age<<"\t"<<st[i].score<<endl;

}

    return 0;
}

```

A structure can also be initialized with values:

```
struct circle c = {12, 23, 5};
```

```
struct circle d = {d.x = 12, d.y = 23, d.r = 5}; // C99
```

An automatic, or local, structure variable can be initialized by

Function call: `struct circle c = circle_init();`

Nested Structure

- You can nest structures within other structures
- A structure may itself contain structures

Example:

```

#include<iostream>
using namespace std;
struct Employee
{
int employee_id;
string  name;
int salary;
};
struct Organisation
{

```

<pre> string orgname; string orglocation; struct Employee emp; }; int main() { struct Organisation org; org.emp.employee_id = 15; org.emp.name="Lemma"; org.emp.salary = 12000; org.orgname="DBU"; org.orglocation= "Debre Berhan"; cout<<org.emp.employee_id<<endl; cout<<org.emp.name<<endl; cout<<org.emp.salary<<endl; cout<<org.orgname<<endl; cout<<org.orglocation; } </pre>	
--	--

Pointer to structure

Pointers to structures may be declared as follow:

struct staructuretage *pintername; eg Employee *emp;

When using a pointer to a struct, member access can be achieved with the `.` operator, but can not look graceful in expression

Example: (*emp).id

Equivalently, the `->' operator can be used; for example: emp->id

<pre> #include<iostream> using namespace std; struct Employee { int empid; string name; int salary; }; </pre>	
---	--

<pre> int main() { Employee *emp,e; emp=&e; (*emp).empid = 15; (*emp).name="Lemma"; (*emp).salary = 12000; cout<<(*emp).empid<<endl; cout<<(*emp).name<<endl; cout<<(*emp).salary<<endl; } </pre>	
---	--

Structure and Function

<pre> #include <iostream> using namespace std; struct Employee { char name[50]; int age; float salary; }; void displayData(Employee); // Function declaration int main() { Employee emp; cout << "Enter Full name: "; cin.get(emp.name, 50); cout << "Enter age: "; cin >> emp.age; cout << "Enter salary: "; cin >> emp.salary; // Function call with structure variable as an argument displayData(emp); return 0; } void displayData(Employee emp) { cout << "\nDisplaying Information." << endl; </pre>	
---	--

<pre>cout << "Name: " << emp.name << endl; cout <<"Age: " << emp.age << endl; cout << "Salary: " << emp.salary; }</pre>	
---	--

NB: A structure can be assigned to, as well as passed to, and returned from functions

A structure declaration cannot contain itself as a member, but it can contain a member which is a pointer whose type is the structure declaration itself

This means we can build recursive data structures; for

Example:

```
struct tree {
    int val;
    struct tree *left;
    struct tree *right;
}
1 struct link {
2     int val;
3     struct link *next;
4 }
```

File and File Management

Files are used to store data in storage devices permanently.

The **fstream** library allows us to work with files.

To use the **fstream** library, include both the standard **<iostream>** **AND** the **<fstream>** header file:

Input/Output with files

- **ofstream:** Stream class to write on files
- **ifstream:** Stream class to read from files

- **fstream**: Stream class to both read and write from/to files.

File Modes

In C++, for every file operation, exists a specific *file mode*. These file modes allow us to *create*, *read*, *write*, *append* or *modify* a file. The file modes are defined in the class **ios**. Let's see all these different **modes** in which we could open a file on disk.

File Modes	Description
ios::in	Searches for the file and opens it in the read mode only(<i>if the file is found</i>).
ios::out	Searches for the file and opens it in the write mode. If the file is found, its content is overwritten. If the file is not found, a new file is created. <i>Allows you to write to the file.</i>
ios::app	Searches for the file and opens it in the append mode i.e. this mode allows you to append new data to the end of a file. If the file is not found, a new file is created.
"ios::binary"	Searches for the file and opens the file(if the file is found) in a binary mode to perform binary input/output file operations.
ios::ate	Searches for the file, opens it and positions the pointer at the end of the file. This mode when used with ios::binary , ios::in and ios::out modes, <i>allows you to modify the content of a file.</i>
"ios::trunc"	Searches for the file and opens it to truncate or deletes all of its content(<i>if the file is found</i>).
"ios::nocreate"	Searches for the file and if the file is not found, a new file will not be created.

Create and Write to a File

To create a file, use either the **ofstream** or **fstream** class, and specify the name of the file. To write to the file, use the insertion operator (**<<**).

Example:

```
#include <iostream>
#include <fstream>
using namespace std;
int main ()
```

```

        { ofstream offile;// File declaration
offile.open("file1.txt");
//offile.open("C:/Users/user/Documents/C++/example1.txt");
offile <<"Testing leads to failure, Failure leads to
understanding!."<<endl;

        offile<<"Good Work";
        offile.close();
        return 0;
    }

```

Read a File

To read from a file, use either the `ifstream` or `fstream` class, and the name of the file. Note that we also use a `while` loop together with the `getline()` function (which belongs to the `ifstream` class) to read the file line by line, and to print the content of the file: To Read from the file, use the Extraction Operator (`>>`).

Example:

```

#include<iostream>
#include<fstream>
using namespace std;
int main()
{ifstream iffile;
iffile.open("file1.txt");
//iffile.open("C:/Users/user/Documents/C++/example1.txt");
string data;
while(getline(iffile,data))
{   cout<<data<<endl;
}
iffile.close();
return 0;}

```

Output:

```

Testing leads to failure,Failure leads to understanding!.
Good Work

```

Example 2:

Write a program that accepts your Name, age and CGPA from the user and store in file name example.txt, and also the program will display the content of the example.txt file for the user.

Solution:

```
#include <iostream>
#include <fstream>
using namespace std;

int main ()
{ string name;
  int age;
  double cgpa;
  ofstream offile;// File declaration
  offile.open("example.txt"); // opening file
  cout<<"Enter Your Name:";
  getline(cin,name);//cin.getline(name,50);
  offile<<name<<endl;
  cout<<"Enter Your Age:";
  cin>>age;
  offile<<age<<endl;
  cout<<"Enter Your CGPA:";
  cin>>cgpa;
  offile<<cgpa<<endl;
  offile <<"Writing this to a file."<<endl;
  offile<<"This is the end";
  offile.close(); // closing file
  return 0;
}
```

Read

```
#include<iostream>
#include<fstream>
using namespace std;
int main()
{ifstream iff;
iff.open("example.txt");
string data;
while(getline(iff,data))
{    cout<<data<<endl;
}
iff.close();
    return 0;
}
```

Employee Payrol System using File

```
#include<iostream>
#include <fstream>
#include<conio.h>
#include <string.h>
#include<stdlib.h>
using namespace std;
struct employee
{
char empid[10];
char empname[10];
double salary;
double tax;
```

```

};
employee emp[100];
fstream readfemp;
fstream writefemp;
int numrecord;
int inrecord;
//calculate tax
double empltax(double salary)
{
    if(salary<=150)
        return 0;
    else if (salary>150 && salary<=600)
        return salary*0.05;
    else if (salary>600 && salary<2000)
        return salary*0.1;
    else if(salary>=2000 && salary <=3000)
        return salary*0.2;
    else
        return salary*0.35;
}
//accept input from keyboard
void input(employee empin[])
{
    char ch='y';
    int i=0;
    while(ch=='y')
    {

```

```

    cout<<"Enter employee id : ";
    cin>>empin[i].empid;
    cout<<"Enter employee name : ";
    cin>>empin[i].empname;
    cout<<"Enter employee salary : ";
    cin>>empin[i].salary;
        empin[i].tax=empltax(empin[i].salary);
        i=i+1;
    cout<<"Do you want to continue n/y ";
    cin>>ch;
}
inrecord=i;
}
//display all records
void display(employee empds[])
{
    int i=0;
        cout<<"Employee id\t Employee name \t Employee salary \n ";
        cout<<"-----
\n";
        while(i<numrecord)
        {
            cout<<empds[i].empid<<"\t\t"<<empds[i].empname<<"\t\t"
<<empds[i].salary<<"\t\t"<<empds[i].tax<<endl;
            i=i+1;
            cout<<endl;
        }
}

```

```

}
int  search(employee empsr[], char sid[])
{
    for(int j=0;j<numrecord;j++)
    {
        if(strcmp(empsr[j].empid,sid)==0)
        {
            return j+1;
        }
    }

return 0;
}

//edit a record
void  editemp(employee empdl[])
{
    writefemp.open("Employee.txt",ios::out);
    int i=0;
    while(i<numrecord)
    {
        writefemp<<empdl[i].empid<<"\t"<<empdl[i].empname<<"\t"
<<empdl[i].salary<<"\t"<<empdl[i].tax<<endl;
        i++;
    }
    writefemp.close();
}

```

```

//delete a record
void deleteemp(employee empdl[], int dlindex)
{
writefemp.open("Employee.txt ",ios::out);
    int i=0;
    while(i<numrecord)
    {
        if(i!=dlindex)
        {
writefemp<<empdl[i].empid<<"\t"<<empdl[i].empname<<"\t"
<<empdl[i].salary<<"\t"<<empdl[i].tax<<endl;
        }
        i++;
    }
    writefemp.close();

}

//read records from a file
void read(employee emprd[])
{
    readfemp.open("Employee.txt ",ios::in);
    int i=0;
    while(!readfemp.eof())
    {
readfemp>>emprd[i].empid>>emprd[i].empname
>>emprd[i].salary>>emprd[i].tax;
        i++;
    }
}

```

```

    numrecord=i-1;
    readfemp.close();
}
//write records to a file
void write(employee empwr[])
{ writefemp.open("Employee.txt",ios::app|ios::out);
    int i=0;
    while(i<inrecord)
    {
        writefemp<<empwr[i].empid<<"\t"<<empwr[i].empname<<"\t"
<<empwr[i].salary<<"\t"<<empwr[i].tax<<endl;
        i++;
    }
    writefemp.close();
}
//main function
int main()
{
int chs;
    system("cls");
    menu:
    cout<<"-----Employee payroll system----- \n";
    cout<<"-----MENU-----\n";
    cout<<"\n\n";
    cout<<"1: Input\n";
    cout<<"2: Display\n";
    cout<<"3: Search\n";
    cout<<"4: Edit\n";

```

```

cout<<"5: Delete\n";
cout<<"6: Exit\n";

cout<<"=====\n";
cin>>chs;
if(chs==1)
{
    input(emp);
    write(emp);
}
else if(chs==2)
{
    read(emp);
    display(emp);
    cout<<"Press any key\n";
    getch();
}
else if(chs==3)
{
    int sindex;
    char srid[10];
    cout<<"Enter employee id \n";
    cin>>srid;
    read(emp);
    sindex=search(emp,srid);
    if(sindex!=0)
        cout<<"employee name is: "<<emp[sindex-1].empname<<endl;
    else

```

```

        cout<<"employee id does not exist\n";
cout<<"Press any key\n";
    getch();
}
else if(chs==4)
{
    int eindex;
    char edid[10];
    cout<<"Enter employee id to edit \n";
    cin>>edid;
    read(emp);
    eindex=search(emp,edid);
    if(eindex!=0)
    {
        cout<<"Enter employee id:\n";
        cin>>emp[eindex-1].empid;
        cout<<"Enter employee name:\n";
        cin>>emp[eindex-1].empname;
        cout<<"Enter employee salary:\n";
        cin>>emp[eindex-1].salary;
        emp[eindex-1].tax=empltax(emp[eindex-1].salary);
        editemp(emp);
        cout<<"employee had edited\n ";
    }
else
    cout<<"Such id does not exist\n";
cout<<"Press any key";

```

```

    getch();
}
else if(chs==5)
{
    int dindex;
    char dlid[10];
    cout<<"Enter employee id to delete \n";
    cin>>dlid;
    read(emp);
    dindex=search(emp,dlid);
    if(dindex!=0)
    {
        deleteemp(emp,dindex-1);
        cout<<"employee had deleted\n ";
    }
    else
        cout<<"Such id does not exist\n";
    cout<<"Press any key";
    getch();
}
else if(chs==6)
{
    goto exit;
}
system("cls");
goto menu;
exit:
cout<<"end of program";

```

```
return 0;  
}
```